Lab 7: Pointers

Computer Programming (010153002) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Address-of, dereference, pass-by-reference, pointers and arrays

Objectives

- Use `&` to take an address and `*` to dereference a pointer.
- Modify a caller's variable from inside a function via a pointer parameter.
- Understand the relationship between arrays and pointers (`a[i]` = `*(a+i)`).
- Recognize common pointer pitfalls: `NULL`, uninitialized pointers, wrong type.

Quick Reference

A pointer holds an address. You can read this as *"p points to x"*.

```
int x = 5;
int *p = &x;         /* p holds the address of x  -- "p points to x" */
*p = 10;             /* write through p; now x == 10 */
printf("%d\n", *p); /* read through p; prints 10 */
```

Pass-by-reference. To let a function modify a caller's variable, pass its address; the function declares a pointer parameter.

```
void inc(int *p) { (*p)++; }
int main(void) { int a = 5; inc(&a); /* a == 6 */ }
```

Arrays and pointers. For `int a[5];` the name `a` *decays* to `&a[0]` in most contexts. So $a[i] \equiv *(a+i) \equiv *(\&a[0]+i)$. Pointer arithmetic is in *elements*, not bytes: `p + 1` skips one `int`.

Pitfalls. `int *p;` alone is uninitialized – dereferencing it is undefined behavior. Always set it (`p = &x;`) or assign `NULL` and check before use.

Quick Experiments (Try It)

Try It 1: Pointer arithmetic – how many bytes does p+1 skip?

Run and record the addresses:

```
#include <stdio.h>
int main(void) {
    int a[3] = {10, 20, 30};
    int *p = a;
    printf("p     = %p,  *p     = %d\n", (void *)p,     *p);
    printf("p+1   = %p,  *(p+1) = %d\n", (void *)(p+1), *(p+1));
    printf("byte gap: %td\n",
           (char *)(p + 1) - (char *)p);
    return 0;
}
```

1. How many bytes did `p+1` advance? Is it the same as `sizeof(int)`?
2. Change `int *p = a;` to `double *pd = (double*)a;` (and use `pd`). What is the byte gap now? What does this tell you about pointer arithmetic?
3. What is the relationship between pointer arithmetic and `sizeof`?

Try It 2: Dereferencing a NULL pointer – see a segfault up close

Save your work first. Then run:

```c
#include <stdio.h>
int main(void) {
    int *p = NULL;        /* points to nothing */
    printf("%d\n", *p);   /* crash here */
    return 0;
}
```

1. What error does the operating system report? (segmentation fault / access violation)
2. Now declare `int *p;` without assigning anything (do not set it to `NULL`). Does it still crash? Is the crash guaranteed or unpredictable?
3. What defensive habit does this experiment motivate?

Try It 3: Two pointers to the same variable

Run and trace the changes:

```c
#include <stdio.h>
int main(void) {
    int x = 1;
    int *p = &x;
    int *q = &x;   /* both p and q point to x */
    *p = 10;
    printf("x = %d, *q = %d\n", x, *q);
    (*q)++;
    printf("x = %d, *p = %d\n", x, *p);
    return 0;
}
```

1. Does writing through `p` also change what `q` reads? Why?
2. Print `&x`, `p`, and `q` with `%p`. Are they equal?
3. This is called *aliasing*. Why can aliasing make code hard to reason about?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Swap via Pointers *(20 min)*

Write `void swap(int *a, int *b)` that swaps the integers its arguments point to. Read two integers in `main`, call `swap(&x, &y)`, and print the result. Why couldn't you write this function with plain `int` parameters? Comment that in your code.

In-Lab Exercise 2: Min/Max via Out-Parameters *(30 min)*

Write `void min_max(int a[], int n, int *out_min, int *out_max)` that finds the minimum and maximum of the array and writes them through the pointer parameters. *One* function, *two* results – pointers are how C returns multiple values.

In-Lab Exercise 3: Pointer Walk over an Array *(35 min)*

Read N doubles into an array and compute the sum using a pointer `p` instead of an index: start with `p = a`, end at `a + N`, increment with `p++`, accumulate `*p`. Print the sum. Compare your code with the index-based version mentally – they should produce the same numbers.

In-Lab Exercise 4: In-Place Reverse with Two Pointers *(35 min)*

Write `void reverse(int *a, int n)` that reverses an array in place using two pointers: `left = a`, `right = a + n - 1`. Swap and walk them toward each other until they meet. Reuse `swap` from Exercise 1.

AI Collaboration

Try these prompts with an AI tool:

- "Explain the difference between `int *p = &x` and `int *p = x` – why does the second one likely crash?"
- "Show me why `void swap(int a, int b)` fails to swap the caller's variables, and fix it with pointers."
- "Draw a text-based memory diagram for: `int x = 5; int *p = &x; *p = 10;`"

Critical check – verify, don't just accept: Ask the AI to write a function using `int *` out-parameters to return two values. Before compiling, check: (a) does every call site pass `&variable`? (b) does the function write through the pointer as `*param = ...`, not `param = ...`? Compile with `-Wall` and confirm zero warnings. Then introduce one deliberate bug (pass a variable without `&`) and verify you get a warning or crash.

Know the limit: AI tools sometimes generate pointer code that is technically correct but introduces aliasing or lifetime bugs that only appear at runtime – always compile with `-Wall` and test with edge inputs, not just the happy path.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Address-of and dereference

What does this print?

```c
#include <stdio.h>
int main(void) {
    int a = 3, b = 7;
    int *p = &a;
    *p = b;
    b = *p + 1;
    printf("%d %d\n", a, b);
    return 0;
}
```

Trace & Predict 2: Array via pointer arithmetic

Predict the exact output:

```c
#include <stdio.h>
int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a + 1;
    printf("%d %d %d\n", *p, *(p + 2), p[-1]);
    return 0;
```

```
}
```

Find the Bug 1: Pass-by-value when pointers were needed

Why doesn't this swap? Fix it.

```c
#include <stdio.h>
void swap(int a, int b) { int t = a; a = b; b = t; }
int main(void) {
    int x = 1, y = 2;
    swap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```

Write Code on Paper 1: Divide with remainder via pointers

Write `void divmod(int a, int b, int *q, int *r)` that stores a/b in `*q` and $a\%b$ in `*r`. Demonstrate from `main`. You may assume $b \neq 0$.

Write Code on Paper 2: Stats via a single pointer pass

Write `void stats(const int *a, int n, int *sum, int *positive_count)` that walks `a` with a pointer and writes both the total sum and the count of strictly positive elements through the out-parameters. Demonstrate from `main`.

Draw on Paper 1: Pointer diagram

Draw a memory diagram for the sequence of statements below. Use **named boxes** for each variable; use **arrows** for pointer relationships. Show the state of every box after *each* assignment (you may draw four snapshots side by side, or annotate a single diagram with step numbers).

```c
int x = 5, y = 10;
int *p = &x;   /* step 1 */
*p = 20;       /* step 2 */
p  = &y;       /* step 3 */
*p = *p + 1;   /* step 4 */
```

Heads up. Pointers reappear in every remaining lab – strings (Lab 8), structs (Lab 9), and files (Lab 10). Pointers are **1/3** of the final exam, so do the take-home in full.